

자연어처리와 컴퓨터 비전 7장 (BERT~)

📅 마감일	@2025년 6월 2일
☑ 발표	☐
☑ 완료	☐
🕒 주차	13주차

BERT (Bidirectional Encoder Representations from Transformers)

- 2018년 구글에서 발표한 언어 모델로 트랜스포머 기반 양방향 인코더를 사용하는 자연어 처리 모델
- 양방향 인코더는 입력 시퀀스를 양쪽 방향에서 처리하여 이전과 이후의 단어를 모두 참조하면서 단어의 의미와 문맥을 파악한다. 기존 언어 모델은 문장을 좌에서 우로 순차적으로 학습해 단어의 의미와 문맥을 파악할 때 이전 단어만 고려할 수 밖에 없었다.
- BERT는 양방향으로 문장을 학습해 이전과 이후의 단어를 모두 참조해 단어의 의미와 문맥을 파악할 수 있기 때문에 기존 모델들보다 더 정확하게 문맥을 파악하고, 다양한 자연어 처리 작업에서 높은 성능을 보인다.
- BERT는 대규모 데이터를 사용해 사전 학습되어 있으므로 전이 학습에 주로 활용된다. BERT는 일부 또는 전체를 다른 작업에서 재사용해 적은 양의 데이터로도 높은 정확도를 달성할 수 있으며, 모델 학습 시간을 단축할 수 있다.

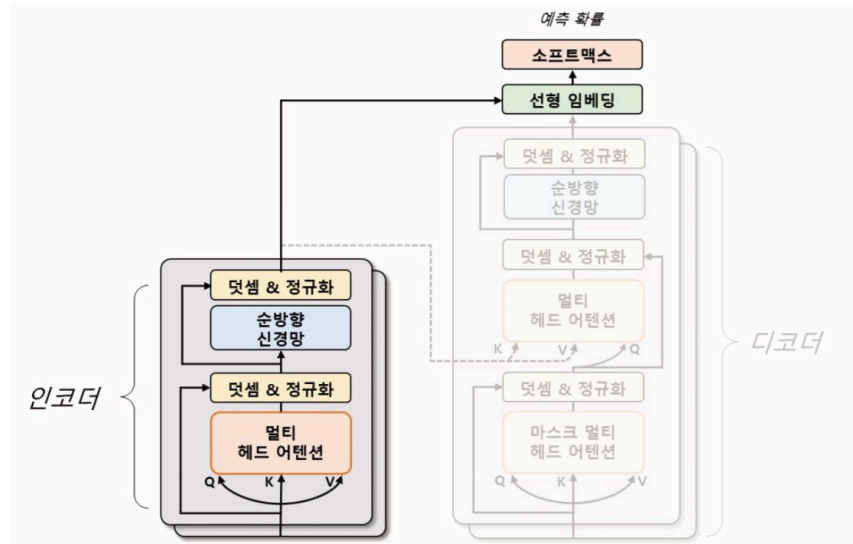


그림 7.13 트랜스포머와 BERT 모델 비교

- BERT는 트랜스포머 인코더 모듈만 사용해 입력 문장을 처리한다.
- BERT는 사전 학습을 위해 마스킹된 언어 모델링(MLM)과 다음 문장 예측 방법(NSP)을 사용한다.

사전 학습 방법

- **MLM**은 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어를 예측하는 방법이다.
 - 예를 들어 I'm learning PyTorch라는 문장에서 learning을 마스킹하면, I'm [MASK] PyTorch가 되어 BERT는 양방향 문맥 정보를 바탕으로 [MASK]에 들어갈 단어를 예측한다.
- BERT는 양방향으로 문장을 학습해 문맥 정보를 모델링하므로 입력 문장에서 누락된 단어를 추론하는 능력을 갖게 된다. 이로써 문장 전체 의미를 이해하는 능력이 향상된다.
- **NSP**는 두 개의 문장이 주어졌을 때, 두 번째 문장이 첫 번째 문장의 다음에 오는 문장인지 여부를 판단하는 작업이다.
 - 예를 들어 I'm learning PyTorch라는 문장과 PyTorch is a machine learning library라는 두 문장이 주어지면 BERT는 이 두 문장이 연속적인 관계인지 아닌지를 예측한다.
 - 이 과정에서 BERT는 두 문장 간 관계를 학습하고, 문장 간의 의미적인 유사성을 파악한다.
- BERT는 입력 문장에서 특수한 토큰들을 추가해 모델이 학습하고 추론하는 과정에서 필요한 정보를 제공한다. [CLS], [SEP], [MASK] 토큰을 사용한다.

- [CLS] 토큰: 입력 문장의 시작 부분에 추가되는 토큰. 이 토큰을 이용해 문장 분류 작업을 위한 정보를 제공한다. 이를 통해 모델은 입력 문장이 어떤 유형의 문장인지 미리 파악할 수 있게 된다.
 - 예를 들어, 이전 문장 분류 작업에서는 이 토큰이 입력 문장이 긍정적인지 부정적인지, 혹은 문장 내에서 어떤 객체가 언급되는지 등을 분류하는데 사용된다.
- [SEP] 토큰: 입력 문장 내에서 두 개 이상의 문장을 구분하기 위해 사용되는 토큰이다.
 - 예를 들어, 문장 분류 작업에서 두 개의 문장을 입력으로 받을 때 [SEP] 토큰을 사용해 두 문장을 구분한다. 이를 통해 모델은 입력 문장을 두 개의 독립적인 문장으로 인식하고, 각각의 문장에 대한 정보를 정확하게 파악할 수 있게 된다.
- [MASK] 토큰: 입력 문장 내에서 임의로 선택된 단어를 가리는 특별한 토큰이다. 주어진 문장에서 일부 단어를 가린 후 모델의 학습과 예측에 활용된다.
 - 예를 들어, 언어 모델링 작업에서 [MASK] 토큰은 마스킹된 실제 단어를 예측하는데 사용된다.
- 따라서 BERT 모델의 입력 시퀀스는 **[CLS] 문장-1 [SEP] 문장-2 [MASK][SEP]** 등의 구조를 가질 수 있다.

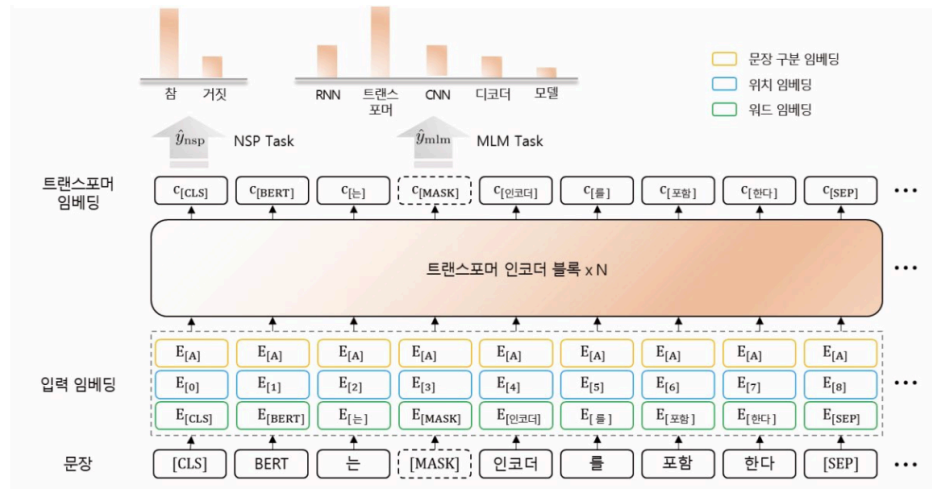


그림 7.14 BERT 모델의 입력 임베딩과 손실 함수

- 위 그림에서 입력 문장은 'BERT는 트랜스포머 인코더를 포함한다' 이다. 그러므로 BERT 모델에서 해당 문장을 처리하기 위해 문장 앞에 [CLS] 토큰과 문장 마지막에 [SEP] 토큰을 추가해 '[CLS] BERT는 트랜스포머 인코더를 포함한다 [SEP]' 과 같은 형태로 변환한다.
- [MASK] 토큰 적용 방법은 텍스트 토큰 중 15%에 해당하는 단어를 대상으로 마스킹을 수행한다. 예를 들어 100 개 토큰 중에서 약 15 개의 단어를 마스킹하고, 그 중 80%는

[MASK] 토큰으로 대체, 나머지 10%는 실제 토큰을 그대로 사용한다.

- BERT는 사전 학습을 수행한 후, 미세 조정 기법을 통해 다양한 자연어 처리 작업에 적용할 수 있다.
 - 예를 들어 문장 분류, 감성 분석, 질문 응답, 기계 번역 등의 다양한 작업이 가능하다. 미세 조정 과정에서는 해당 작업에 맞는 계층을 추가하고 손실 함수를 정의해 학습한다.
- 분류 문제에서는 일반적으로 [CLS] 토큰 벡터를 사용하고, 각 토큰마다 예측이 필요한 경우 모든 토큰 벡터를 사용한다.

모델 실습

7.15 네이버 영화 리뷰 데이터 불러오기

```
import numpy as np
import pandas as pd
from Korpora import Korpora

corpus = Korpora.load("nsmc")

df = pd.DataFrame(corpus.test).sample(20000, random_state=42)

train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(train.head(5).to_markdown())
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

```
...
| 18010 | 징키스칸이란 소재를 가지고 이것밖에 못만드나 | 0 |
Training Data Size : 12000
Validation Data Size : 4000
Testing Data Size : 4000
```

7.16 BERT 입력 텐서 생성

```

import torch
from Korpora import Korpora
from transformers import BertTokenizer
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, Seq

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 5
batch_size = 32
device = "cuda" if torch.cuda.is_available() else "cpu"

tokenizer = BertTokenizer.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    do_lower_case=False
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)

```

```
# 6. 확인용 출력
print(train_dataset[0])
```

- BERT 토크나이저 클래스는 사전 학습된 토크나이저를 불러와 전처리를 수행한다. 사전 학습된 모델은 `bert-base-multilingual-cased` 로 다중 언어를 지원하며 대소문자를 유지하는 사전 학습된 BERT 모델을 의미한다. 소문자 유지(`do_lower_case`) 매개변수를 `False` 로 할당해 소문자로 변환하지 않게 한다.
 - 소문자 유지 매개 변수를 `True` 로 설정하면 'Apple'과 'apple'을 다른 단어로 인식하는 방법으로 학습하게 된다.
- 이 토크나이저를 `make_dataset` 함수에 전달해 텐서 데이터셋을 반환한다. 출력 결과에서 텍스트는 숫자 ID와 어텐션 마스크로 변경되며, 레이블이 텐서 형식으로 변환된 것을 확인할 수 있다.
- `get_data_loader` 함수로 샘플러 클래스를 활용해 데이터를 목적에 따라 샘플링한다.

```
from torch import optim
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)
```

```
optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- BERT 모델도 GPT-2 문장 분류 모델처럼 BERT 문장 분류 모델 (`BertForSequenceClassification`) 클래스로 불러올 수 있다. BERT 모델은 패딩 기법을 사용하므로 모델 설정을 변경하지 않는다.
- 활성화 함수는 `AdamW` 로, Adam 최적화 함수에 가중치 감쇠를 추가한 변형된 경사 하강법 알고리즘이다. AdamW 클래스의 엡실론(`eps`) 매개변수는 학습률을 0으로 나누는 것을 방지하기 위해 분모에 더하는 작은 값이다.
- BERT 모델은 대규모의 매개변수를 가진 딥러닝 모델이므로 안정적인 기울기 갱신이 필수적이다. `AdamW` 는 갱신되는 모든 매개변수의 학습률을 조절하는 적응형 활성화 함수이므로 모델이 빠르게 수렴하고 안정적으로 학습할 수 있다.

BERT 모델 구조 출력 결과

```
from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("  L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("    L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("      L", sssub_name)
```

```

bert
├── L embeddings
│   ├── L word_embeddings
│   ├── L position_embeddings
│   ├── L token_type_embeddings
│   ├── L LayerNorm
│   └── L dropout
├── L encoder
│   ├── L layer
│   │   ├── L 0
│   │   ├── L 1
│   │   ├── L 2
│   │   ├── L 3
│   │   ├── L 4
│   │   ├── L 5
│   │   ├── L 6
│   │   ├── L 7
│   │   ├── L 8
│   │   ├── L 9
│   │   ├── L 10
│   │   └── L 11
├── L pooler
│   ├── L dense
│   └── L activation
└── dropout
    └── classifier

```

- BERT 모델의 입력 임베딩은 토큰 임베딩(`word_embeddings`), 위치 임베딩(`position_embeddings`), 문장 구분 임베딩(`token_type_embeddings`)으로 구성된다.
- 이러한 임베딩들은 서로 다른 정보를 담고 있으며, 이를 결합해 입력 시퀀스에 대한 임베딩 벡터를 생성한다. 이후 생성된 임베딩 벡터는 계층 정규화와 드롭아웃을 수행한다.
- 트랜스포머 인코더 블록은 총 12개를 사용하며, 각각의 블록은 입력 시퀀스를 순차적으로 처리해 새로운 임베딩을 생성한다. 이 과정에서 각 블록은 셀프 어텐션 및 순방향 신경망을 사용해 입력 시퀀스의 다양한 관계를 학습한다.
- 풀러(`pooler`)는 `[CLS]` 토큰 벡터를 한 번 더 비선형 변환을 수행하기 위해 선형 변환과 비선형 변환인 `Tanh` 함수를 사용한다. 이후, 드롭아웃이 적용돼 모델의 과대적합을 방지한다. 마지막에 적용되는 분류기(`classifier`)는 BERT 모델에서 수행해야 하는 작업으로, `[CLS]` 토큰 벡터를 활용해 결과를 예측한다.

BERT 모델 학습

```

import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

```



```

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    train_loss = train_loss / len(dataloader)
    return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        criterion = nn.CrossEntropyLoss()
        val_loss, val_accuracy = 0.0, 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        logits = outputs.logits

        loss = criterion(logits, labels)
        logits = logits.detach().cpu().numpy()

```

```

        label_ids = labels.to("cpu").numpy()
        accuracy = calc_accuracy(logits, label_ids)

        val_loss += loss.item()
        val_accuracy += accuracy

    val_loss = val_loss / len(dataloader)
    val_accuracy = val_accuracy / len(dataloader)
    return val_loss, val_accuracy

best_loss = 10000

for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)

    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy: {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "models/BERTForSequenceClassification.pt")
        print("Saved the model weights")

```

```

Epoch 1: Train Loss: 0.6943 Val Loss: 0.6935 Val Accuracy: 0.4930
Saved the model weights
Epoch 2: Train Loss: 0.6946 Val Loss: 0.6935 Val Accuracy: 0.4930
Epoch 3: Train Loss: 0.6934 Val Loss: 0.6935 Val Accuracy: 0.4930
Epoch 4: Train Loss: 0.6933 Val Loss: 0.6935 Val Accuracy: 0.4930
Epoch 5: Train Loss: 0.6948 Val Loss: 0.6935 Val Accuracy: 0.4930

```

BERT 모델 평가

```

from transformers import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="bert-base-multilingual-cased",
    num_labels=2
).to(device)

```

```

model.load_state_dict(torch.load("models/BERTForSequenceClassification.pt"))

test_loss, test_accuracy = evaluation(model, test_dataloader)

print(f"Test Loss : {test_loss:.4f}")
print(f"Test Accuracy : {test_accuracy:.4f}")

```

Test Loss : 0.6933
Test Accuracy : 0.4975

BART (Bidirectional Auto-Regressive Transformer)

- 2019년 메타 연구소에서 발표한 트랜스포머 기반 모델이다.
- BART는 BERT 인코더와 GPT의 디코더를 결합한 시퀀스-시퀀스 구조로 **노이즈 제거 오토인코더(Denoising Autoencoder)**로 사전 학습된다. 오토인코더 사전 학습 방법은 입력 데이터에 잡음을 추가하고, 잡음이 없는 원본 데이터를 복원하도록 학습하는 방식으로 수행된다.
- BERT 인코더는 입력 문장에서 일부를 무작위로 마스킹해 처리하고, 마스킹된 단어를 맞추도록 학습하여 문장 전체의 맥락을 이해하고 문맥 내 단어간 상호작용을 파악할 수 있다.
- 반면, GPT는 언어 모델을 이용해 문장의 이전 토큰들을 입력으로 받고, 다음에 올 토큰을 맞추도록 학습한다. 이를 통해 GPT는 문장 내 단어들의 순서와 문맥을 파악하며, 다음에 올 단어를 예측하는 능력을 갖게 된다.
- BART는 사전 학습시 BERT 인코더와 GPT 디코더가 학습하는 방법을 일반화해 학습한다.

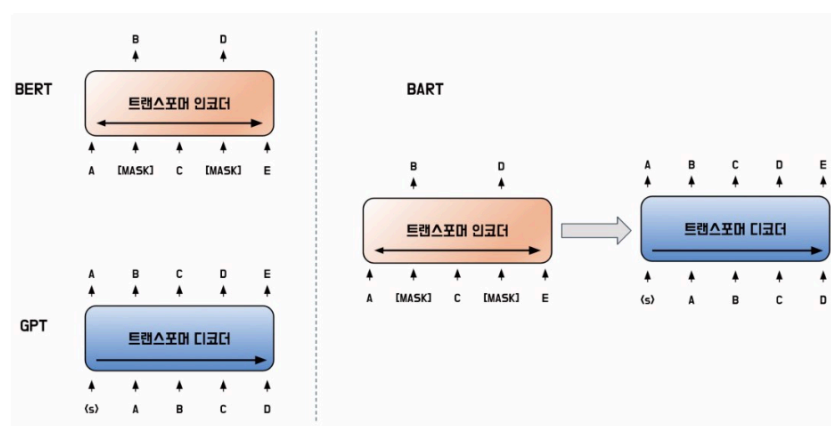


그림 7.15 BART 사전 학습 시각화

- BART 사전 학습 방법은 위 그림과 같다. BART는 사전 학습시 노이즈 제거 오토인코더를 사용하므로 입력 문장에 임의로 노이즈를 추가하고 원래 문장을 복원하도록 학습한다. 노이즈가 추가된 텍스트를 인코더에 입력하고 원본 텍스트를 디코더에 입력해 디코더가 원본 텍스트를 생성할 수 있게 학습한다. 이는 BERT가 인코더 학습시 텍스트에 마스킹을 추가해 노이즈를 주는 방법과 GPT가 디코더 학습시 다음에 등장할 단어를 예측하는 방법으로 간주할 수 있다. 이를 통해 BART는 문장 구조와 의미를 보존하면서도 다양한 변형을 학습할 수 있다. 이 구조는 입력 문장에 큰 제약 없이 노이즈 기법을 적용할 수 있으므로 더 풍부한 언어적 지식을 습득할 수 있게 한다.
- 인코더로 순방향 정보만 인식할 수 있는 GPT 단점을 개선 (양방향 문맥 정보 반영), 디코더로 문장 생성 분야에서 뛰어나지 않았던 BERT의 단점 개선

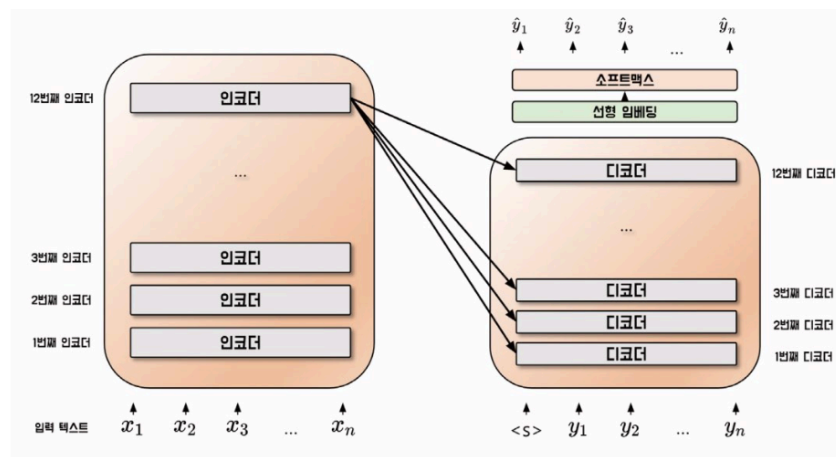


그림 7.16 BART 모델 구조

- BART의 모델 구조는 다음과 같다. BART는 인코더와 디코더를 모두 사용하므로 트랜스포머와 유사한 구조를 가지고 있다. 하지만, 트랜스포머는 인코더의 모든 계층과 디코더의 모든 계층 사이의 어텐션 연산을 수행했다면, BART는 인코더 마지막 계층과 디코더 각 계층 사이에만 어텐션 연산을 수행한다.
- BART 인코더는 입력 문장의 각 단어를 임베딩하고 여러 층의 인코더를 거쳐 마지막 계층에서는 입력 문장 전체의 의미를 가장 잘 반영하는 벡터가 생성된다. 이렇게 생성된 벡터는 디코더가 출력 문장을 생성할 때 참고되며, 디코더의 각 계층에서는 이 벡터와 이전 계층에서 생성된 출력 문장의 정보를 활용해 출력 문장을 생성한다.
- BART에서는 인코더의 마지막 계층과 디코더 마지막 계층 사이에서만 어텐션 연산을 수행하므로 정보 전달을 최적화하고 메모리 사용량을 줄일 수 있다.

사전 학습 방법

- BART 인코더는 BERT의 MLM 이외에도 다양한 노이즈 기법을 사용한다. 문장의 일부를 가리는 토큰 마스킹 기법 이외에도 토큰 삭제, 문장 교환, 텍스트 채우기 기법이 사용

된다. 다음은 BART 사전 학습에 사용되는 여러 노이즈 기법을 시각화한 것이다.



그림 7.17 BART 노이즈 기법

- **토큰 마스크:** MLM과 동일한 기법으로 입력 문장의 일부 토큰을 마스크 토큰으로 치환하는 방법. 문장 내에서 어떤 단어가 중요한 역할을 하는지 학습할 수 있으며, 입력 문장의 문맥을 이해하고 문장 내에서 중요한 정보를 추출하는 능력이 향상된다.
- **토큰 삭제:** 입력 문장의 일부 토큰을 치환하는 것이 아니라, 삭제하는 방법. 토큰 마스크와 다르게 어떤 위치의 토큰이 삭제되었는지도 맞춰야한다. 이를 이용하면 입력 문장에서 불필요한 정보나 중요하지 않은 정보를 자동으로 필터링해 처리할 수 있게 되며, 모델의 학습과 예측 시간을 줄이고, 모델의 일반화 성능을 향상시킬 수 있다.
- **문장 순열:** 마침표(.)를 기준으로 문장을 나눈 뒤, 문장의 순서를 섞는 방법이다. 모델은 원래의 문장 순서를 맞춰야한다. 이 기법은 문장 내에서 단어들이 어떻게 연결되어 있는지를 더 잘 파악하게 되며 입력 문장이 다른 순서로 주어졌을 때에도 잘 처리할 수 있게 된다.
- **문서 회전:** 임의의 토큰으로 문서가 시작하도록 한다. 문장 순열과는 다르게 문장 순서는 유지한다. 모델은 문서의 원래 시작 토큰을 맞춰야한다. 이 기법은 모델이 문서의 시작점을 인식할 수 있도록 한다.
- **텍스트 채우기:** 몇 개의 토큰을 하나의 구간으로 묶고 일부 구간을 마스크 토큰으로 대체한다. 묶은 구간의 길이는 0부터 임의의 값까지 설정할 수 있으며, 일반적으로 포아송 분포로 구간을 나누며 λ 를 3으로 설정해 계산한다. 구간 길이가 0인 경우 해당 위치에 변화된 토큰이 없음을 의미한다.
- 모델은 연속된 마스크 토큰을 복구하되, 실제로는 마스크되지 않은 토큰도 구분해야한다. 이를 통해 모델이 누락된 단어를 예측하게 유도함으로써 더 많은 정보를 활용해 입력 문장을 더 잘 이해할 수 있게 된다.

미세 조정 방법

- BART는 인코더와 디코더를 모두 사용하는 구조를 가지고 있기 때문에 미세 조정 시 각 다운스트림 작업에 맞게 입력 문장을 구성해야한다. 즉, 인코더와 디코더에 다른 문장 구조로 입력한다.

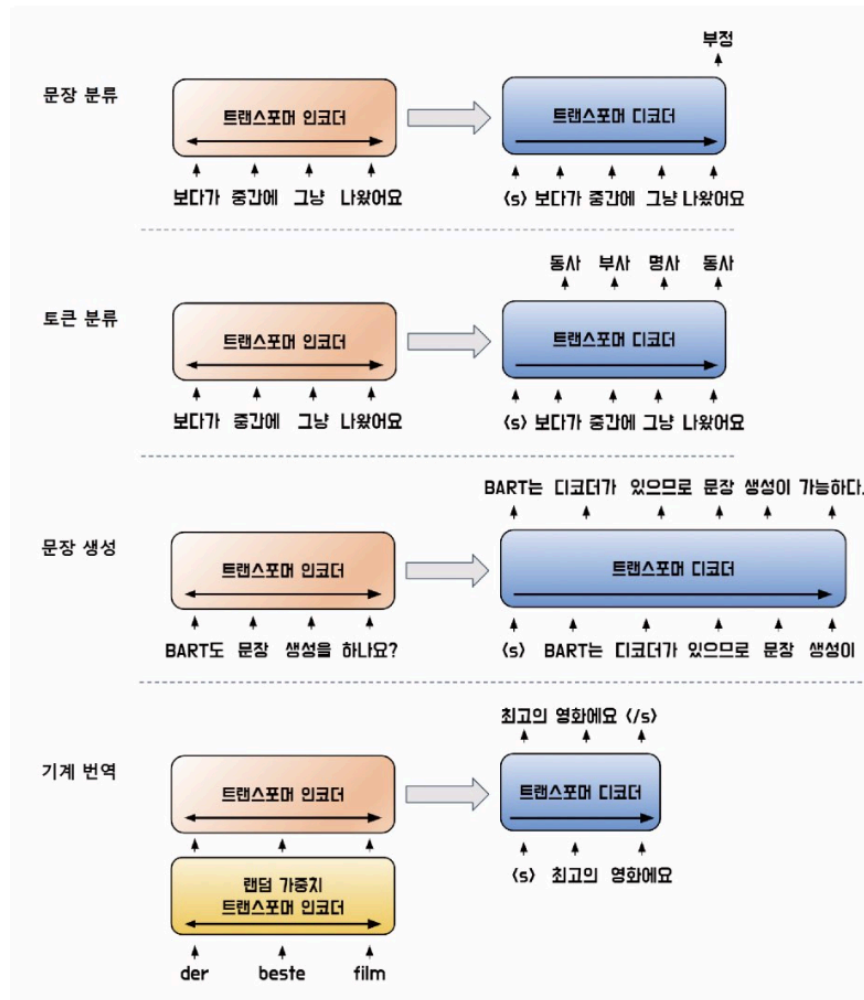


그림 7.18 BART 미세 조정 구조

- 미세 조정 방법은 다음과 같다.
- 문장 분류 작업**에선 입력 문장을 인코더와 디코더에 동일하게 입력. 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 사용한다. BERT의 CLS 토큰과 비슷하지만, BART는 *전체 입력과 어텐션 연산이 적용된 은닉 상태* 사용
- 토큰 분류 작업에서도 입력 문장을 인코더와 디코더에 동일하게 입력. 디코더의 각 시점 별 마지막 은닉 상태를 토큰 분류기의 입력값으로 사용한다. 전체 입력과 디코더의 각 시점 별 은닉 상태와의 어텐션 연산을 수행한다.

- BART는 트랜스포머 디코더를 사용하기 때문에 BERT가 해결하지 못 했던 문장 생성 작업을 수행할 수 있다. 특히 입력값을 조작해 출력을 생성하는 **추상적 질의 응답 (Abstractive Question Answering)**과 **문장 요약(Summarization)**과 같은 작업에 적절하다.
- 이러한 작업들은 BART의 사전 학습 방식과 유사하기 때문에 뛰어난 성능을 보인다. 이러한 학습 방법으로 BART는 문장의 의미를 파악하고, 이를 기반으로 새로운 문장을 생성하는 능력을 갖게 된다.
 - 예를 들어 기계 번역 작업에서 BART는 사전 학습된 인코더에 기계 번역을 위한 인코더를 추가해 작업을 수행할 수 있다. 이때 추가된 인코더는 기존의 단어 사전을 사용하지 않아도 되며, 디코더는 사전 학습된 가중치와 단어 사전을 사용한다.
- 학습 단계는 두 단계로 나뉜다. 첫 번째 단계에서는 새로 추가된 트랜스포머 인코더의 가중치와 위치 임베딩, 그리고 첫 번째 인코더 층의 셀프 어텐션 입력 행렬 가중치만 학습한다. 두 번째 단계에서는 모든 신경망의 가중치를 작은 반복으로 학습한다.
- BART는 문장 내부의 토큰 사이의 상관 관계를 파악해 문장의 의미를 더욱 정확하게 이해할 수 있다. 입력 문장의 전체적인 의미를 고려하므로 BART가 자연어 생성 분야에서는 매우 효과적이다.

모델 실습

- 허깅 페이스 라이브러리의 BART 모델과 뉴스 요약 데이터셋을 활용해 문장 요약 모델을 미세 조정해본다. 뉴스 요약 데이터셋은 미국의 인공지능 회사 아르길라에서 공개한 데이터셋으로 뉴스 본문과 본문의 요약 텍스트로 구성된다.

7.18 뉴스 요약 데이터셋 불러오기

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])

train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(f"Source News : {train.text.iloc[0][:200]}")
```

```
print(f"Summarization : {train.prediction.iloc[0][:50]}")
print(f"Training Data Size : {len(train)}")
print(f"Validation Data Size : {len(valid)}")
print(f"Testing Data Size : {len(test)}")
```

```
Source News : DANANG, Vietnam (Reuters) - Russian President Vladimir Putin said on Saturday he had a
normal dialogue with U.S. leader Donald Trump at a summit in Vietnam, and described Trump as civil,
well-educated
Summarization : Putin says had useful interaction with Trump at Vi
Training Data Size : 3000
Validation Data Size : 1000
Testing Data Size : 1000
```

- 뉴스 요약 데이터세트는 뉴스 본문과 이를 짧게 요약한 짧은 텍스트로 구성되며, 모델은 뉴스 본문을 입력으로 받아 요약된 텍스트를 출력해야한다.
- 문장 요약 작업은 문장 길이가 긴 텍스트를 다루기 때문에 리뷰 분류와 같이 상대적으로 짧은 텍스트를 다루는 작업에 비해 연산량이 많으므로 데이터셋을 5000개만 샘플링해 사용한다. 샘플링한 데이터셋을 6:2:2 비율로 학습, 검증 및 테스트 데이터로 분리해 사용한다.

7.19 BART 입력 텐서 생성

```
import torch
from transformers import BartTokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )

    labels = []
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
```



```

for target in data.prediction:
    labels.append(tokenizer.encode(target, return_tensors="pt").squeeze())

labels = pad_sequence(labels, batch_first=True, padding_value=-100).to(device)
return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader

epochs = 3
batch_size = 8
device = "mps" if torch.backends.mps.is_available() else "cpu"

tokenizer = BartTokenizer.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

```

(
  tensor([ 0, 495, 1889, ..., 1, 1, 1], device='cuda:0'),
  tensor([1, 1, 1, ..., 0, 0, 0], device='cuda:0'),
  tensor([ 0, 35891, 161, 56, 5616, 10405, 19, 140, 23, 5490,
          3564, 2, -100, -100, -100, -100, -100, -100, -100, -100,
          -100, -100, -100, -100, -100, -100, -100, -100, -100, -100], device='cuda:0')
)

```

- BART 토크나이저도 BERT 토크나이저 클래스와 동일하게 사전 학습된 토크나이저를 불러와 전처리를 수행한다. 사전 학습된 모델은 `facebook/bart-base` 로 BART 모델의 기본

버전을 의미한다.

- 요약 작업은 입/출력이 모두 텍스트 데이터이다. 문장 길이는 각각 다르기 때문에 길이를 맞춰주기 위해 패딩을 적용한다. 이때 패딩값은 -100을 사용한다. 이는 교차 엔트로피와 같은 손실 함수에서 패딩된 토큰을 무시하게 하기 위해 사용한다. 패딩된 토큰은 모델의 출력과 비교하지 않고 실제 레이블을 가진 토큰만 손실을 계산하는데 사용한다.
- 데이터로더는 요약하려는 본문 텍스트의 정수 인코딩 값, 어텐션 마스크, 요약 문장의 정수 인코딩 값을 변환한다. 데이터로더까지 적용했다면 모델과 최적화 함수를 선언한다.

7.20 BART 모델 선언

```
from torch import optim
from transformers import BartForConditionalGeneration

model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path='facebook/bart-base'
).to(device)
optimizer = optim.AdamW(model.parameters(), lr = 5e-5, eps = 1e-8)
```

- BART 조건부 생성 클래스(`BartForConditionalGeneration`)는 BART 모델의 변형 중 하나로 조건부 생성 작업에 특화된 모델이다. 이 모델은 입력 시퀀스로부터 출력 시퀀스를 생성하는데 사용된다. 문장 요약, 기계 번역, 질의 응답 등과 같은 작업에 사용될 수 있다.
- 빠른 학습을 위해 12개의 인코더/디코더 계층이 아닌 6개의 계층을 사용하는 `facebook/bart-base` 모델을 사용한다. (`facebook/bart-large` 모델로 12개 계층 사용 가능)

```
from transformers import BartForConditionalGeneration

model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base",
    num_labels=2
).to(device)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
```

```
for sssub_name, sssub_module in ssub_module.named_children():
    print("| | L", sssub_name)
```

```
model
├── L shared
│   ├── L encoder
│   │   ├── L embed_tokens
│   │   ├── L embed_positions
│   │   ├── L layers
│   │   │   ├── L 0
│   │   │   ├── L 1
│   │   │   ├── L 2
│   │   │   ├── L 3
│   │   │   ├── L 4
│   │   │   └── L 5
│   │   └── L layernorm_embedding
│   └── L decoder
│       ├── L embed_tokens
│       ├── L embed_positions
│       ├── L layers
│       │   ├── L 0
│       │   ├── L 1
│       │   ├── L 2
│       │   ├── L 3
│       │   ├── L 4
│       │   └── L 5
│       └── L layernorm_embedding
└── lm_head
```

- BART는 인코더와 디코더가 동일한 임베딩 계층을 사용한다. **shared** 계층은 인코더와 디코더가 공유하는 임베딩 계층을 의미하며, 이러한 공유로 인코더와 디코더 간의 연결을 강화한다.
- BART는 6개의 인코더와 디코더 계층으로 구성되며, 마지막 연산에서는 **layernorm_embedding** 계층을 통과한다. **layernorm_embedding** 계층은 인코더와 디코더에서 각 토큰의 임베딩에 적용되는 계층 정규화로 임베딩 벡터의 마지막 차원에 대해 정규화를 수행해 학습을 안정화한다.
- 인코더의 마지막 계층의 출력값은 디코더의 모든 계층과 어텐션 연산을 수행한다. 마지막 디코더 계층의 출력값은 출력 크기가 단어 사전의 크기인 완전 연결 계층을 통과해 언어 모델을 형성한다.

예제 7.21 BART 모델 학습 및 평가

- BART 모델 평가 방법은 문장 생성 기법에서 자주 사용되는 **루지 (Recall-Oriented Understudy for Gisting Evaluation, ROUGE)** 점수를 사용한다. 루지 점수는 생성된 요약문과 정답 요약문이 얼마나 유사한지를 평가하기 위해 토큰의 N-gram 정밀도와 재현율을 이용해 평가하는 지표다.
 - 예를 들어, 유니그램을 사용하는 점수는 ROUGE-1, 바이그램을 사용하는 점수는 ROUGE-2, N-gram을 사용하면 ROUGE-N이라고 한다.

정답 문장 유니그램	[대한민국, 은, 16, 강, 에, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$
정답 문장 바이그램	[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$

그림 7.19 루지 점수 계산 방법

- ROUGE-N 이외에도 ROUGE-L, ROUGE-LSUM, ROUGE-W 등이 있다.
 - **ROUGE-L**은 생성된 요약문과 정답 요약문 사이에서 **최장 공통부분 수열** (Longest Common Subsequence, **LCS**) 기반의 통계 방식이다.
 - LCS는 두 문장 사이에 공통으로 존재하는 가장 긴 부분 문자열을 찾는 문제로, 문장의 구조적 유사성을 고려하고 가장 길게 연속되는 N-gram을 식별한다. 이 방법은 요약 문장이 입력 문장의 의미를 잘 전달하는지를 평가하는 데 유용하다.
 - **ROUGE-LSUM**은 ROUGE-L의 변형으로 텍스트 내의 개행 문자를 문장 경계로 인식하고, 각 문장 쌍에 대해 LCS를 계산한 후, union-LCS라는 값을 계산한다. union-LCS는 각 문장 쌍의 LCS를 합집합한 것으로, 중복된 부분을 제거한 후 길이를 계산한다. ROUGE-LSUM은 텍스트 생성 작업에서 요약의 정확성과 완전성을 모두 반영할 수 있는 지표로 사용된다.
 - **ROUGE-W**는 가중치가 적용된 LCS(Weighted LCS-based) 방법으로 연속된 LCS에 가중치를 부여해 계산한다. 이 방법은 공통부분 문자열의 길이뿐만 아니라 해당 부분 문자열 내의 단어에 가중치를 부여해 평가하는 방식이다. 이 방법은 단어 간 유사도를 고려해 요약 문장이 입력 문장의 의미를 더욱 잘 전달하는지 평가하는 데 유용하다.

```
import numpy as np
import torch
from transformers import BartForConditionalGeneration
```

```

from torch import nn
from tqdm import tqdm

# ROUGE 평가 로드
import evaluate
rouge_score = evaluate.load("rouge")

def calc_rouge(preds, labels):
    preds = preds.argmax(axis=-1)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)

    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    rouge2 = rouge_score.compute(
        predictions=decoded_preds,
        references=decoded_labels
    )
    return rouge2["rouge2"]

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for input_ids, attention_mask, labels in tqdm(dataloader):
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss
        train_loss += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    return train_loss / len(dataloader)

```

```

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss, val_rouge = 0.0, 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss
        logits = outputs.logits

        logits = logits.detach().cpu().numpy()
        label_ids = labels.to("cpu").numpy()
        rouge = calc_rouge(logits, label_ids)

        val_loss += loss
        val_rouge += rouge

    val_loss /= len(dataloader)
    val_rouge /= len(dataloader)

    return val_loss, val_rouge

model = BartForConditionalGeneration.from_pretrained("facebook/bart-base")
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-5)

best_loss = float("inf")
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_rouge = evaluation(model, valid_dataloader)

    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f}")

    if val_loss < best_loss:

```

```
best_loss = val_loss
torch.save(model.state_dict(), "./bart_best.pt")
print("Saved the model weights")
```

출력 결과

```
Epoch 1: Train Loss: 2.1677 Val Loss: 1.8385 Val Rouge 0.2564
Saved the model weights
Epoch 2: Train Loss: 1.6036 Val Loss: 1.9068 Val Rouge 0.2608
Epoch 3: Train Loss: 1.2503 Val Loss: 1.9909 Val Rouge 0.2538
```

- `calc_rouge` 함수는 텍스트 요약 작업에서 모델이 예측한 요약문과 정답 요약문 사이의 루지 점수를 계산하는 함수다.
- `preds` 는 모델이 예측한 요약의 토큰 인덱스를 담은 2차원 배열이므로 `argmax` 메서드를 통해 각 토큰에 대해 가장 높은 확률을 가진 인덱스를 선택하여 1차원 배열로 변경한다. `labels` 는 정답 요약문으로, 레이블이 -100인 값들을 패딩 토큰의 인덱스로 변경한다.
- 토큰라이저의 `batch_decode` 메서드로 특수 토큰을 제외하고 토큰 인덱스를 실제 텍스트로 변환한다. 이 값을 이용해 루지(`rouge_score`) 인스턴스의 컴퓨팅(`compute`) 메서드로 루지 점수를 계산한다. `rouge2` 변수는 다음과 같은 형태로 반환된다.

rouge2 출력 결과

```
{
  'rouge1': 0.44116997177658945,
  'rouge2': 0.2,
  'rougeL': 0.4275670306001189,
  'rougeLsum': 0.4243807560903149
}
```

```
model = BartForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="facebook/bart-base"
).to(device)

model.load_state_dict(torch.load("models/BartForConditionalGeneration.pt"))

test_loss, test_rouge_score = evaluation(model, test_dataloader)

print(f"Test Loss : {test_loss:.4f}")
print(f"Test ROUGE-2 Score : {test_rouge_score:.4f}")
```

출력 결과

Test Loss : 1.7890

Test ROUGE-2 Score : 0.2699

- ROUGE-2는 0에서 1 사이의 값을 가지며, 1에 가까울수록 높은 성능을 의미한다. 모델 학습 시 매우 작은 크기의 데이터로 샘플링해 학습을 수행했음을 고려한다면 중간 수준의 성능을 보인다고 할 수 있다.
- 문장 요약 작업에서는 수치화된 평가 지표만으로는 어느 정도로 잘 요약했는지 판단하기가 어렵다. 앞선 7.2 'GPT' 절의 예제 7.10에서 사용한 파이프라인 함수를 활용해 문장을 요약한 후 예측된 요약문과 정답 요약문을 비교해 본다.
- 이를 통해 요약문이 입력 문장과 얼마나 유사하게 요약됐는지 직접 확인할 수 있다.

7.23 문장 요약문 비교

```
from transformers import pipeline

summarizer = pipeline(
    task="summarization",
    model=model,
    tokenizer=tokenizer,
    max_length=54,
    device="cpu"
)

for index in range(5):
    news_text = test.text.iloc[index]
    summarization = test.prediction.iloc[index]
    predicted_summarization = summarizer(news_text)[0]["summary_text"]

    print(f"정답 요약문 : {summarization}")
    print(f"모델 요약문 : {predicted_summarization}\n")
```


정답 요약문 : Clinton leads Trump by 4 points in Washington Post: ABC News poll
 모델 요약문 : Clinton leads Trump by 4 percentage points in Washington Post-ABC poll

정답 요약문 : Democrats question independence of Trump Supreme Court nominee
 모델 요약문 : U.S. senators question Supreme Court nominee Gorsuch's independence

정답 요약문 : In push for Yemen aid, U.S. warned Saudis of threats in Congress
 모델 요약문 : U.S. warns Saudi Arabia over Yemen humanitarian situation

정답 요약문 : Romanian ruling party leader investigated over 'criminal group'
 모델 요약문 : Romanian prosecutors investigate leader of ruling party over graft

정답 요약문 : Billionaire environmental activist Tom Steyer endorses Clinton
 모델 요약문 : Steyer backs Clinton for U.S. president

- BART는 BERT의 한계를 극복하기 위해 인코더와 디코더 구조를 사용하여 문장 생성 작업에서 높은 성능을 발휘한다. BART는 문장 요약과 같은 다양한 작업에서 뛰어난 성능을 보이며, 이를 가능케 하는 여러 가지 노이즈 기법을 사용한다.
- 예를 들어, BART는 마스킹된 언어 모델을 비롯하여 문장 내 단어의 순서를 섞는 등의 방법을 사용하여 사전 학습한다. 이러한 다양한 방법을 통해 BART는 다양한 자연어 처리 작업에서 탁월한 성능을 보인다.

ELECTRA(Efficiently Learning an Encoder that Classifies Token Replacements Accurately)

- 2020년 구글에서 발표한 트랜스포머 기반의 모델이다.
- BERT를 비롯해 많은 자연어 처리 모델은 마스킹된 언어 모델링 방식을 사용하거나 일부 토큰을 [MASK]로 대체하여 입력을 손상시키고 원래 토큰을 복원하는 모델을 학습한다.
- 하지만 **ELECTRA**는 입력을 마스킹하는 대신 **생성자(Generator)**와 **판별자(Discriminator)**를 사용해 사전 학습을 수행한다. ELECTRA의 사전 학습 접근 방식은 변환기 모델인 생성자와 판별자를 학습하므로 생성적 적대 신경망(GAN)과 유사한 방법으로 학습이 수행된다.
 - 생성 모델은 실제 데이터와 비슷하게 토큰을 생성해 다른 토큰으로 대체하고 판별 모델이 생성 모델이 만든 데이터와 실제 데이터를 입력 받아 어떤 데이터가 실제 데이터인지, 아니면 생성된 데이터인지 구분한다.
- ELECTRA는 GAN을 사용해 학습하므로 이전 언어 모델과 비교해 더 효율적인 학습이 가능하다. 덕분에 대규모 데이터셋에서 모델을 더 빠르게 학습할 수 있다. 생성 모델을 통해 토큰을 생성하므로 다양한 자연어 생성 작업에서 보다 자연스러운 문장을 생성하게 된다.

- 또한 BERT와 같은 모델과 비교했다면 모델의 매개 변수 수가 더 작다. 그로 인해 모델의 크기가 줄어들어 모델을 더 빠르게 실행할 수 있으며, 더 적은 메모리를 사용한다.

사전 학습 방법

- ELECTRA의 생성자 모델과 판별자 모델은 모두 트랜스포머 인코더 구조를 따른다. 생성자 모델은 입력 문장의 일부 토큰을 마스크 처리하고 처리된 토큰이 원래 어떤 토큰이었는지 예측하며 학습한다.
- 반면 판별자 모델은 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 인해 바뀐 토큰인지 맞히며 학습한다. 이러한 학습 방법을 **RTD(Replaced Token Detection)**라고 한다.

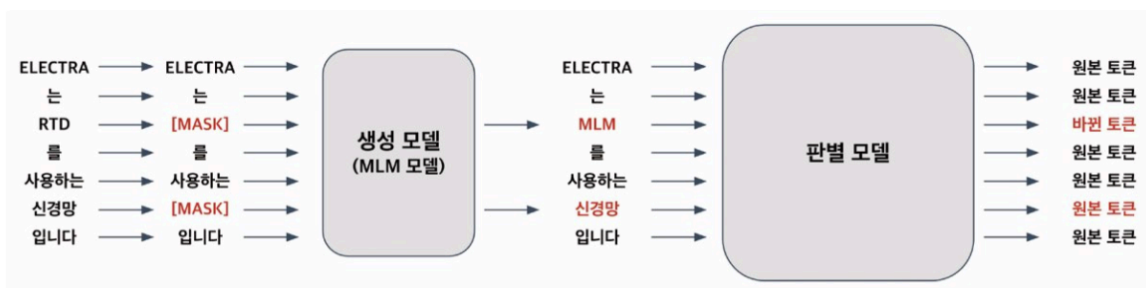


그림 7.20 ELECTRA 모델 학습 방법

- 생성자 모델은 BERT의 마스킹된 언어 모델과 동일하다. 입력 문장의 15%를 마스크 처리해 마스크 처리된 토큰이 원래 어떤 토큰이었는지를 맞히며 학습한다.
- 판별자 모델은 생성자 모델이 복원한 결과값을 입력 받아 각 토큰이 원본 토큰인지 바뀐 토큰인지를 판별한다. 이는 GAN과 유사하지만, 생성자 모델이 원래 토큰을 정확히 예측한 경우 이 토큰은 생성된 토큰이 아닌 원본 토큰으로 인식한다.
- 위 그림에서 생성 모델이 두 번째 마스크 토큰을 '신경망'으로 원본과 동일하게 생성했다.
 - GAN에서는 원본이 아닌 생성된 것으로 간주하지만, ELECTRA는 원본 토큰으로 간주한다.
 - 또, GAN은 생성 모델과 판별 모델을 적대적으로 학습한다. 생성 모델은 판별 모델이 실제 데이터와 구분하기 어렵게 학습하고, 판별 모델은 실제 데이터와 생성된 데이터를 더욱 잘 구분하게 학습한다.
 - 하지만 ELECTRA 생성 모델은 마스킹된 언어 모델을 통해 학습되며, 판별 모델은 각 토큰이 바뀐 토큰인지 아니면 원본 토큰인지 구분하도록 학습한다.
 - GAN은 완전한 노이즈 벡터를 입력 받아 생성하지만, ELECTRA 생성 모델은 일부 토큰이 마스크 처리된 텍스트를 입력으로 받는다.

- 생성 모델과 판별 모델은 모두 트랜스포머 인코더 구조를 따르므로 두 모델이 같은 계층 개수를 가지고 있다면 가중치를 공유할 수 있어 더 빠르게 학습할 수 있다. 그러나 두 모델이 가중치를 완전히 공유하면 생성 모델 성능이 너무 높아져 판별 모델이 학습할 수 없게 되므로 ELECTRA는 판별 모델 크기를 1/2에서 1/4로 크기로 바꿔 설정하고 모든 가중치를 공유하는 대신 임베딩 계층의 가중치만 공유한다. 이를 통해 판별 모델이 학습하기 적합한 생성 모델을 유지하면서도 가중치를 공유하여 효율적으로 학습할 수 있다.
- ELECTRA는 사전 학습이 완료되면 생성 모델을 사용하지 않고 오직 판별 모델만 사용해 다운 스트림 작업을 수행한다. 판별 모델은 트랜스포머 인코더로 구성되어 있으며, BERT와 동일한 구조를 갖는다. 다운 스트림 작업은 BERT와 동일한 방식으로 미세 조정한다.

모델 실습

- 허깅 페이스 라이브러리의 ELECTRA 모델과 네이버 영화 리뷰 감정 분석 데이터셋으로 모델을 학습한다.

7.24 네이버 영화 리뷰 데이터셋 전처리

```
import torch
from Korpora import Korpora
from transformers import ElectraTokenizer
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, Seq

def make_dataset(data, tokenizer, device):
    tokenized = tokenizer(
        text=data.text.tolist(),
        padding="longest",
        truncation=True,
        return_tensors="pt"
    )
    input_ids = tokenized["input_ids"].to(device)
    attention_mask = tokenized["attention_mask"].to(device)
    labels = torch.tensor(data.label.values, dtype=torch.long).to(device)
    return TensorDataset(input_ids, attention_mask, labels)

def get_dataloader(dataset, sampler, batch_size):
    data_sampler = sampler(dataset)
    dataloader = DataLoader(dataset, sampler=data_sampler, batch_size=batch_size)
    return dataloader
```

```

epochs = 5
batch_size = 32
device = "mps" if torch.backends.mps.is_available() else "cpu"

tokenizer = ElectraTokenizer.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    do_lower_case=False
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(train_dataset[0])

```

```

(tensor([ 2, 6511, 14347, 4087, 4665, 4112, 2924, 4806, 16, 3809,
4309, 4275, 16, 3201, 4376, 2891, 4139, 4212, 4007, 6557,
4200, 5, 5, 3, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0], device='mps:0'), tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0], device='mps:0'), tensor(1, device='mps:0'))

```

- 영어 텍스트 분류를 위해 만들어진 ELECTRA 모델과 한국어 텍스트 분류를 위해 만들어진 KoELECTRA가 제공된다.
- 영어 텍스트 분류는 `google/electra-small` 과 `google/electra-base` , `google/electra-large` 로 적용할 수 있으며, 한국어 텍스트 분류는 `monologg/koelectra-small-v3` 과 `monologg/electra-base-v3` 로 적용할 수 있다. 이번 실습에서는 `koelectra-base` 모델을 사용한다.
- ELECTRA는 판별 모델만을 이용해 다운 스트림 작업을 수행하므로 `koelectra-base` 모델의 판별 모델을 의미하는 `monologg/koelectra-base-v3-discriminator` 을 불러온다. 생성 모델을 불러와야 하는 경우 `monologg/koelectra-base-v3-generator` 로 불러올 수 있다.

7.25 KoELECTRA 모델 선언

```
from torch import optim
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    num_labels=2
).to(device)

optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

- 문장 분류를 위해 문장 분류 모델 `ElectraForSequenceClassification` 클래스를 불러온다. BERT 모델과 동일한 최적화 함수와 매개 변수를 사용한다.

모델 구조 출력 결과

```
from transformers import ElectraForSequenceClassification

model = ElectraForSequenceClassification.from_pretrained(
    pretrained_model_name_or_path="monologg/koelectra-base-v3-discriminator",
    num_labels=2
).to(device)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
```

```
for sssub_name, sssub_module in ssub_module.named_children():
    print("| | L", sssub_name)
```

```
electra
L embeddings
|   L word_embeddings
|   L position_embeddings
|   L token_type_embeddings
|   L LayerNorm
|   L dropout
L encoder
|   L layer
|   |   L 0
|   |   L 1
|   |   L 2
|   |   L 3
|   |   L 4
|   |   L 5
|   |   L 6
|   |   L 7
|   |   L 8
|   |   L 9
|   |   L 10
|   |   L 11
classifier
L dense
L activation
L dropout
L out_proj
```

- KoELECTRA는 12개의 인코더 계층을 통과해 입력 텍스트 상태 값을 반환한다. 분류기 계층은 문장 분류를 위한 계층으로 [CLS] 토큰 벡터를 이용해 입력 텍스트에 대한 분류를 수행한다.

KoELECTRA 모델 학습 및 평가

```
import numpy as np
from torch import nn

def calc_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

def train(model, optimizer, dataloader):
```

```

model.train()
train_loss = 0.0

for input_ids, attention_mask, labels in dataloader:
    outputs = model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )

    loss = outputs.loss
    train_loss += loss.item()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

train_loss = train_loss / len(dataloader)
return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        criterion = nn.CrossEntropyLoss()
        val_loss, val_accuracy = 0.0, 0.0

    for input_ids, attention_mask, labels in dataloader:
        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        logits = outputs.logits

        loss = criterion(logits, labels)
        logits = logits.detach().cpu().numpy()
        label_ids = labels.to("cpu").numpy()
        accuracy = calc_accuracy(logits, label_ids)

```

```

        val_loss += loss.item()
        val_accuracy += accuracy

    val_loss = val_loss / len(dataloader)
    val_accuracy = val_accuracy / len(dataloader)
    return val_loss, val_accuracy

best_loss = 10000

for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss, val_accuracy = evaluation(model, valid_dataloader)

    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f} Val Accuracy: {val_accuracy:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "models/ElectraForSequenceClassification.pth")
        print("Saved the model weights")

```

KoELECTRA모델 학습 결과

```

Epoch 1: Train Loss: 0.4489 Val Loss: 0.3227 Val Accuracy 0.8715
Saved the model weights
Epoch 2: Train Loss: 0.2891 Val Loss: 0.2928 Val Accuracy 0.8808
Saved the model weights
Epoch 3: Train Loss: 0.2188 Val Loss: 0.3111 Val Accuracy 0.8802

```

KoELECTRA모델 평가 결과

```

Test Loss : 0.3084
Test Accuracy : 0.8732

```

- 모델 평가 결과 정확도는 87.32%로 7.3 'BERT'의 정확도인 80.85%에 비해 약 7.5% 높은 것을 확인할 수 있다.
- 자연어 처리 분야에서 모델을 평가할 때 가장 보편적인 방법은 GLUE(General Language Understanding Evaluation) 벤치마크(Benchmark) 데이터셋을 사용하는 것이다.

- GLUE는 문장 수준 또는 문서 수준의 이해력을 평가하는 데이터세트로, 문장 분류, 문장 유사도 계산, 자연어 추론, 질의응답 등 11가지 과제를 통해 모델의 성능을 평가한다.
- ELECTRA는 GLUE 평가에서 높은 점수를 기록한다. electra-small과 bert-small은 동일한 구조와 가중치 개수를 가지지만, electra-small은 bert-small보다 5% 높은 점수를 기록했다.
- 비슷한 방식으로, electra-base와 bert-base는 약 3% 차이, electra-large와 bert-large는 약 2% 차이로 ELECTRA 모델이 더 좋은 점수를 기록했다.
- 또한 12시간 학습한 electra-small 모델은 bert-small 모델이 4일 학습 결과보다 더 좋은 GLUE 점수를 기록했다. 이는 ELECTRA가 더 효율적으로 학습되어 더 높은 성능을 보인다는 것을 의미한다.

T5

- **T5(Text-to-Text Transfer Transformer)**는 2019년 구글에서 발표한 자연어 처리 분야의 딥러닝 모델로 트랜스포머 구조를 기반으로 한다.
 - T5는 인코더-디코더 모델 구조를 바탕으로 GLUE, SuperGLUE, CNN/DM(Cable News Network/Daily Mail) 등의 데이터세트에서 **SOTA(State-of-the-art)**를 달성했으며 다양한 자연어 처리 작업에서 높은 성능을 보이는 모델이다.
- 기존의 자연어 처리 모델은 대부분 입력 문장을 벡터나 행렬로 변환한 뒤, 이를 이용해 출력 문장을 생성하는 방식이거나 출력값이 클래스 또는 입력값의 일부를 반환하는 형식으로 동작했지만, T5는 입력과 출력을 모두 토큰 시퀀스로 처리하는 **텍스트-텍스트(Text-to-Text)** 구조다. 따라서 입력과 출력의 형태를 자유롭게 다룰 수 있으며, 모델 구조상 유연성과 확장성이 뛰어나기 때문에 새로운 자연어 처리 작업에서도 쉽게 적용할 수 있다.
 - 이러한 특징 덕분에 T5는 자연어 처리 분야에서 다양한 작업에 사용할 수 있다. 대표적인 학습 작업으로는 문장 번역, 요약, 질의응답, 텍스트 분류 등이 있다. 다음 그림 7.21은 T5 모델 학습에 사용한 작업들을 보여준다.

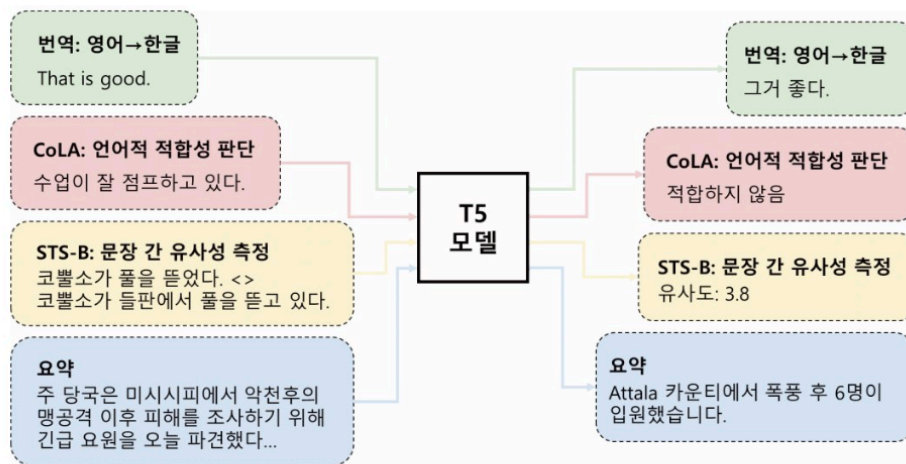


그림 7.21 T5 모델 학습 유형

- T5는 입력과 출력이 **모두 토큰(텍스트) 시퀀스**이기 때문에 입력과 출력 간의 관계를 더욱 세밀하게 다룰 수 있다. T5는 사전 학습 후 미세 조정 단계에서 해당 작업의 데이터를 이용해 모델을 조정해 최적의 성능을 얻을 수 있다.
- CoLA 데이터세트를 사용하면 문법적으로 허용 가능한 문장과 그렇지 않은 문장을 구분할 수 있으며, **STS-B(The Semantic Textual Similarity Benchmark)** 데이터세트를 사용하면 문장 간 의미적 유사성을 측정할 수 있게 된다.
- T5는 입력과 출력을 모두 텍스트 시퀀스로 처리하는 텍스트-텍스트 모델이므로 학습을 위한 데이터세트는 원본 문장과 대상 문장을 사용한다. T5는 C4(Colossal Clean Crawled Corpus) 데이터세트를 활용해 다양한 자연어 처리 작업을 수행할 수 있게 사전 학습됐다.
- **사전 학습 방식은 비지도 학습 방식**으로 입력 문장의 일부 구간을 마스킹해 입력 시퀀스를 처리하며, 출력 시퀀스는 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성된다.
- 이때 문장마다 유일한 마스크 토큰을 의미하는 센티넬 토큰(Sentinel Token)이 사용된다. 센티넬 토큰은 `<extra_id_0>`, `<extra_id_1>` 과 같이 0부터 99까지의 100개의 값이 사용된다.
 - 예를 들어 '인코더-디코더 모델 구조'라는 문장에서 '인코더'와 '디코더'를 마스킹해 처리할 경우, 입력 토큰의 센티넬 토큰은 `[<extra_id_0>, -, <extra_id_1>]`, 모델 구조로 생성되며, 출력 토큰은 `[인코더, <extra_id_0>, 디코더, <extra_id_1>]` 로 구성된다.
- **T5의 미세 조정은 지도 학습 방식**으로 학습된다. 번역, 언어 수용성, 문장 유사도, 문서 요약 등 다양한 작업에 활용할 수 있다. T5 모델의 입력과 출력은 텍스트 시퀀스 토큰으로 이루어져 있으며, 인코더-디코더 T5 모델에 입력된다.

- 이 과정에서 문장이 인코딩되기 전에 'translate English to German:' 또는 'summerize:'와 같은 작업 토큰을 문장 앞에 추가한다. 이러한 방식은 작업 토큰도 함께 학습해 다양한 자연어 처리 작업에서 높은 성능을 발휘할 수 있게 된다.

모델 실습

7.26 뉴스 요약 데이터 세트 불러오기

```
import numpy as np
from datasets import load_dataset

news = load_dataset("argilla/news-summary", split="test")
df = news.to_pandas().sample(5000, random_state=42)[["text", "prediction"]]
df["text"] = "summarize: " + df["text"]
df["prediction"] = df["prediction"].map(lambda x: x[0]["text"])

train, valid, test = np.split(
    df.sample(frac=1, random_state=42), [int(0.6 * len(df)), int(0.8 * len(df))]
)

print(f"Source News : {train.text.iloc[0][:200]}")
print(f"Summarization : {train.prediction.iloc[0][:50]}")
```

```
Source News : summarize: DANANG, Vietnam (Reuters) - Russian President Vladimir Putin said on
Saturday he had a normal dialogue with U.S. leader Donald Trump at a summit in Vietnam, and described
Trump as civil, we
Summarization : Putin says had useful interaction with Trump at Vi
```

- `text`: 뉴스 본문, `prediction`: 요약된 뉴스

7.27 뉴스 요약 데이터세트 전처리

```
import torch
from transformers import T5Tokenizer
from torch.utils.data import TensorDataset, DataLoader
from torch.utils.data import RandomSampler, SequentialSampler
from torch.nn.utils.rnn import pad_sequence

def make_dataset(data, tokenizer, device):
```

```

source = tokenizer(
    text=data.text.tolist(),
    padding="max_length",
    max_length = 128,
    pad_to_max = True,
    truncation=True,
    return_tensors="pt"
)

target = tokenizer(
    text = data.prediction.tolist(),
    padding = "max_length",
    max_length = 128,
    pad_to_max = True,
    return_tensors = "pt"
)

source_ids = source['input_ids'].squeeze().to(device)
source_mask = source['attention_mask'].squeeze().to(device)
target_ids = target['input_ids'].squeeze().to(device)
target_mask = target['attention_mask'].squeeze().to(device)

return TensorDataset(source_ids, source_mask, target_ids, target_mask)

epochs = 3
batch_size = 2
device = "mps" if torch.backends.mps.is_available() else "cpu"

tokenizer = T5Tokenizer.from_pretrained(
    pretrained_model_name_or_path="t5-small"
)

train_dataset = make_dataset(train, tokenizer, device)
train_dataloader = get_dataloader(train_dataset, RandomSampler, batch_size)

valid_dataset = make_dataset(valid, tokenizer, device)
valid_dataloader = get_dataloader(valid_dataset, SequentialSampler, batch_size)

```

```

test_dataset = make_dataset(test, tokenizer, device)
test_dataloader = get_dataloader(test_dataset, SequentialSampler, batch_size)

print(next(iter(train_dataloader)))
print(tokenizer.convert_ids_to_tokens(21603))
print(tokenizer.convert_ids_to_tokens(10))

```

```

[tensor([[21603, 10, 283, ..., 11, 16, 1],
        [21603, 10, 8747, ..., 4008, 65, 1],
        [21603, 10, 549, ..., 1384, 3, 1],
        ...,
        [21603, 10, 309, ..., 34, 845, 1],
        [21603, 10, 549, ..., 0, 0, 0],
        [21603, 10, 3, ..., 5, 1, 0]], device='cuda:0'),
 tensor([[1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 1, 1, 1],
        ...,
        [1, 1, 1, ..., 1, 1, 1],
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 1, 1, 0]], device='cuda:0'),
 tensor([[ 8994, 28167, 4775, ..., 0, 0, 0],
        [ 3, 31, 279, ..., 0, 0, 0],
        [ 1945, 1384, 3, ..., 0, 0, 0],
        ...,

```

```

        [11279, 13849, 22895, ..., 0, 0, 0],
        [ 1945, 1384, 845, ..., 0, 0, 0],
        [ 2968, 6323, 20143, ..., 0, 0, 0]], device='cuda:0'),
 tensor([[1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0],
        ...,
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0],
        [1, 1, 1, ..., 0, 0, 0]], device='cuda:0')]
 _summarize
 :
```

- **T5 토큰나이저(T5Tokenizer)** 클래스도 BERT 토큰나이저 클래스와 동일하게 사전 학습된 토큰나이저를 불러와 전처리를 수행한다. 사전 학습된 모델은 t5-small로 T5 모

델의 기본 버전을 의미한다.

- 이번 예제의 패딩 방식은 `pad_to_max_length` 매개변수로 최대 길이(`max_length`)보다 짧으면 패딩을 수행하고 길면 문장을 자른다. 128 길이보다 긴 경우 128로 맞춰지며, 길이가 128보다 작은 경우 패딩이 수행된다.
- 뉴스 본문과 요약된 뉴스에 토큰 인덱스(`input_ids`)와 어텐션 마스크(`attention_mask`)를 반환하고 토큰화 결과를 출력한다.
- 토큰 인덱스 앞에 반복되는 21603과 10은 뉴스 본문 앞에 붙인 'summarize:'를 의미한다. 토큰라이저의 `convert_ids_to_tokens` 메서드로 토큰의 출력값을 확인할 수 있다.

7.28 T5 모델 선언

```
from torch import optim
from transformers import T5ForConditionalGeneration

model = T5ForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="t5-small",
).to(device)

optimizer = optim.AdamW(model.parameters(), lr=1e-5, eps=1e-8)
```

모델 선언

```
from transformers import T5ForConditionalGeneration

model = T5ForConditionalGeneration.from_pretrained(
    pretrained_model_name_or_path="t5-small",
    num_labels=2
).to(device)

for main_name, main_module in model.named_children():
    print(main_name)
    for sub_name, sub_module in main_module.named_children():
        print("L", sub_name)
        for ssub_name, ssub_module in sub_module.named_children():
            print("| L", ssub_name)
            for sssub_name, sssub_module in ssub_module.named_children():
                print("| | L", sssub_name)
```

```

shared
encoder
  L embed_tokens
  L block
  | L 0
  | | L layer
  ...
  | L 5
  | | L layer
  L final_layer_norm
  L dropout
decoder
  L embed_tokens
  L block
  | L 0
  | | L layer
  ...
  | L 5
  | | L layer
  L final_layer_norm
  L dropout
lm_head

```

- T5 모델은 인코더와 디코더 함수의 토큰 임베딩을 서로 공유함

7.29 T5 모델 학습 및 평가

```

import numpy as np
from torch import nn

def train(model, optimizer, dataloader):
    model.train()
    train_loss = 0.0

    for source_ids, source_mask, target_ids, target_mask in dataloader:
        decoder_input_ids = target_ids[:, :-1].contiguous()
        labels = target_ids[:, 1:].clone().detach()
        labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100

        outputs = model(
            input_ids=source_ids,

```

```

        attention_mask=source_mask,
        decoder_input_ids=decoder_input_ids,
        labels=labels,
    )

    loss = outputs.loss
    train_loss += loss.item()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

train_loss = train_loss / len(dataloader)
return train_loss

def evaluation(model, dataloader):
    with torch.no_grad():
        model.eval()
        val_loss = 0.0

    for source_ids, source_mask, target_ids, target_mask in dataloader:
        decoder_input_ids = target_ids[:, :-1].contiguous()
        labels = target_ids[:, 1:].clone().detach()
        labels[target_ids[:, 1:] == tokenizer.pad_token_id] = -100

        outputs = model(
            input_ids=source_ids,
            attention_mask=source_mask,
            decoder_input_ids=decoder_input_ids,
            labels=labels,
        )

        loss = outputs.loss
        val_loss += loss

    val_loss = val_loss / len(dataloader)
    return val_loss

```



```
# Training loop
best_loss = 10000
for epoch in range(epochs):
    train_loss = train(model, optimizer, train_dataloader)
    val_loss = evaluation(model, valid_dataloader)

    print(f"Epoch {epoch + 1}: Train Loss: {train_loss:.4f} Val Loss: {val_loss:.4f}")

    if val_loss < best_loss:
        best_loss = val_loss
        torch.save(model.state_dict(), "../models/T5ForConditionalGeneration.pt")
        print("Saved the model weights")
```

```
Epoch 1: Train Loss: 4.3135 Val Loss: 3.3195
Saved the model weights
Epoch 2: Train Loss: 3.4098 Val Loss: 2.9162
Saved the model weights
Epoch 3: Train Loss: 3.1287 Val Loss: 2.7619
Saved the model weights
```

- T5 모델은 토큰 인덱스(`input_ids`), 어텐션 마스크(`attention_mask`), 디코더 토큰 인덱스(`decoder_input_ids`), 라벨(`labels`)로 모델을 학습한다.
- `decoder_input_ids` 는 디코더에 입력될 시퀀스를 인코딩한 텐서로, 마지막 토큰을 제외한 나머지 토큰을 사용하고, `labels` 는 `decoder_input_ids` 보다 다음 시점을 예측하도록 `target_ids[:, 1:]` 로 설정한다. 이때 `pad_token_id` 토큰에 대해서는 `-100` 으로 설정해 손실값 계산 시 무시되게 설정한다.
- `decoder_input_ids` 변수에 적용된 `contiguous` 메서드는 텐서를 메모리상에 연속된 블록으로 저장하는 역할을 한다. 이를 통해 메모리상에 인접한 위치에 저장된 데이터를 빠르게 연산할 수 있다.
- 출력 결과를 보면 매우 작은 데이터로 모델을 학습했음에도 불구하고 학습 손실과 검증 손실이 점차 감소하는 것을 확인할 수 있다.

```
model.eval()
with torch.no_grad():
    for source_ids, source_mask, target_ids, target_mask in test_dataloader:
        generated_ids = model.generate(
            input_ids=source_ids,
```

```

        attention_mask=source_mask,
        max_length=128,
        num_beams=3,
        repetition_penalty=2.5,
        length_penalty=1.0,
        early_stopping=True,
    )

    for generated, target in zip(generated_ids, target_ids):
        pred = tokenizer.decode(
            generated, skip_special_tokens=True, clean_up_tokenization_spaces=True
        )
        actual = tokenizer.decode(
            target, skip_special_tokens=True, clean_up_tokenization_spaces=True
        )
        print("Generated Headline Text:", pred)
        print("Actual Headline Text :", actual)
        break

```

```

Generated Headline Text: Clinton leads Trump by 4 percentage points in four-war race for Nov. 8
election.
Actual Headline Text   : Clinton leads Trump by 4 points in Washington Post: ABC News poll
Generated Headline Text: U.S. senators sharpen potential line of attack against Gorsuch's nomination
to Supreme Court
Actual Headline Text   : Democrats question independence of Trump Supreme Court nominee
Generated Headline Text: Saudi-led coalition pushed Riyadh to allow greater access for humanitarian
aid. U.S. warns about Yemen humanitarian situation could constrain U.S. assistance, official says.
Actual Headline Text   : In push for Yemen aid, U.S. warned Saudis of threats in Congress
...

```

- **생성(generate)** 메서드는 입력 문장(입력 시퀀스)에 대한 요약문(출력 시퀀스)을 생성한다. 토큰 인덱스(`input_ids`)와 어텐션 마스크(`attention_mask`)는 입력 문장의 인코딩과 마스킹 정보를 나타내며, 최대 길이(`max_length`)는 생성될 요약문의 최대 길이를 의미한다.
- **빔 개수(num_beams)**는 **빔 서치(Beam Search)** 알고리즘의 빔 크기를 의미한다. 빔 서치 알고리즘은 디코더 모델이 생성한 다수의 후보 단어 시퀀스 중에서 가장 높은 확률을 가진 시퀀스를 선택해 출력한다.
- 디코더 모델은 다음 단어를 예측하는 과정에서 다수의 후보 단어를 생성한다. 이때 빔 서치 알고리즘은 미리 지정한 빔 크기만큼의 후보 단어 시퀀스만을 유지하고, 나머지 후보

시퀀스들은 삭제한다. 이후 다음 단어를 예측하면서 빔 크기에 맞게 후보 시퀀스들을 업데이트하며, 최종적으로 가장 높은 확률을 가진 시퀀스를 선택한다.

- 예를 들어, 빔 크기가 3이라면, 디코더 모델이 생성한 후보 시퀀스 중에서 가장 높은 확률을 가진 상위 3개의 시퀀스를 선택하고, 이후에는 이 3개의 시퀀스만을 유지하면서 다음 단어를 예측한다. 이를 반복해 빔 크기에 맞게 선택된 후보 시퀀스 중에서 가장 높은 확률을 가진 시퀀스를 출력으로 사용한다.

- **반복 페널티(`repetition_penalty`)**는 중복 토큰 생성을 제어하는 값이다. 이 값이 높을수록 중복 토큰 생성이 억제된다.
 - **길이 페널티(`length_penalty`)**는 생성된 시퀀스 길이에 대한 보상을 제어한다. 이 값이 높을수록 더 긴 시퀀스가 생성된다.
 - **조기 중단(`early_stopping`)**은 최대 길이에 도달하기 전 EOS 토큰이 생성되는 경우 중단한다.
 - 생성 메서드로 입력 시퀀스에 대한 출력 시퀀스를 생성할 수 있으며, 생성된 토큰 시퀀스를 토큰라이저의 **디코딩(`decode`)** 메서드로 디코딩할 수 있다.
 - `skip_special_tokens` 와 `clean_up_tokenization_spaces` 매개변수는 디코딩된 텍스트에서 특수 토큰과 불필요한 공백을 제거한다.
-
- 출력 결과를 보면 T5 모델이 생성한 요약 결과와 실제 요약 결과가 비슷한 내용을 전달하고 있음을 알 수 있다. 하지만 요약 내용이 완전히 일치하지는 않는다. 더 정확한 요약을 위해서 더 많은 학습 데이터와 하이퍼파라미터 튜닝으로 모델 성능을 개선할 수 있다.